HOTEL BOOKING AND OCCUPANCY MANAGEMENT SYSTEM

Java-Based PBL Project

IKJOT KAUR MATHARU • HARSHINI RACHURI • SRI BHAVANA

Team No: 18

PSPP • PROBLEM SOLVING USING PROGRAMMING (JAVA)

ABSTRACT

A Java-based hotel management system that organizes room booking, cancellation, customer details, occupancy tracking and billing.

INPUT

Customer name, room number, number of days and menu choice.

PROCESS

Check room status → book/cancel → update occupancy → calculate bill.

OUTPUT

Room status, booking confirmation, occupancy rate and total bill.

Project flow: Rooms → Bookings → Customers → Occupancy → Billing

UNDERSTANDING THE PROBLEM STATEMENT

EXISTING PROBLEMS

- Double booking of rooms
- Difficulty tracking vacant rooms
- Calculation mistakes
- Time-consuming record management
- Difficulty finding booking details

PROPOSED SOLUTION

Build a Java program that keeps room status and booking information organized.

The system uses simple menu-driven operations to connect booking, occupancy and billing.

Goal: The problem statement of our project
Hotel Booking & Occupancy Management System
is to address the difficulties in **managing hotel rooms and bookings manually**.

PROJECT OBJECTIVES & FEATURES

ROOM MANAGEMENT

View available and occupied rooms.

CUSTOMER MANAGEMENT

Store customer name and booking details.

BOOKING MANAGEMENT

Book and cancel rooms safely.

OCCUPANCY

Track total, occupied and available rooms.

BILLING

Calculate room charges and total amount.

ACCURACY

Reduce manual errors and avoid double booking.

CO 1 — MODULE 1: COMPUTATIONAL THINKING

From problem to program: define the problem before writing Java code.

COMPUTATION

Input → Process → Output

For the project:
Input: customer/room/days
Process: availability + billing
Output: booking + bill

THINKING PILLS

- Decomposition
- Pattern recognition
- Abstraction
- Algorithm design

Used to break hotel operations into smaller tasks.

PROBLEM SPECIFICATION

Identify:
- Inputs and outputs
- Assumptions
- Constraints
- Edge cases

Example edge case: room already occupied.

CO 1 — MODULE 1: JAVA FOUNDATIONS

ALGORITHMS & JAVA

- Pseudocode and flowcharts
- Stepwise refinement
- Desk-checking and trace tables
- Pre/post-conditions and invariant idea

One problem can have many correct algorithms.

JAVA TOOLCHAIN

Source code → javac compiler → bytecode → JVM

Minimal Java program: class, main, statements and blocks.

Concept: “write once, run anywhere”.

DATA & OPERATORS

Primitive types: byte, short, int, long, float, double, char, boolean.

Variables, literals, constants (final), arithmetic, relational, logical, assignment and ternary operators.

EXPRESSIONS & I/O

Precedence, associativity, mixed-type arithmetic, casting, integer division, overflow and floating-point imprecision.

Console I/O with Scanner; readable names and comments.

CO 2 — MODULE 2: SELECTION & DECISION MAKING

IF / ELSE

Choose actions based on conditions.

Example:

```
if (!occupied[room])
   book room;
else
   show “occupied”.
```

Nested if-else and if-else-if ladders handle multiple conditions.

SWITCH

Useful for menu-driven operations.

1. View Rooms
2. Book Room
3. Cancel Booking
4. Check Occupancy
5. Exit

switch selects the matching menu case.

BOOLEAN LOGIC

Boolean conditions use && and ||.

Short-circuit evaluation avoids unnecessary checks.

Hotel example:
room valid AND room not occupied.

Decision tables can map complex multi-condition logic before coding.

CO 2 — MODULE 2: ITERATION & LOOP CONTROL

LOOPS

while • do-while • for • enhanced for-each

Loop anatomy:
initialisation → condition → body → update

Use definite loops when the number of repetitions is known.

PROJECT USE

Loop through room data to:
- display rooms
- count occupied rooms
- count available rooms
- search for a room
- calculate totals

Patterns: counting, summing, max/min, search and accumulation.

CONTROL & SAFETY

Counter, sentinel and flag-controlled loops.

break → stop the loop
continue → skip current iteration

Termination reasoning helps avoid infinite loops and off-by-one errors.

Nested loops support table/grid processing and two-dimensional reasoning.

CO 3 — MODULE 3: METHODS & RECURSION

METHODS

Named, reusable abstractions.

Declaration → signature → invocation

Parameters and arguments:
formal vs actual

Return value vs void

Least scope keeps variables where they are needed.

DESIGN

Method overloading allows same method name with different parameter lists.

Single-responsibility decomposition creates a clear call hierarchy.

Project methods can include:
bookRoom(), cancelRoom(), calculateBill(), showOccupancy().

RECURSION

A recursive method has:
- Base case
- Recursive case

Classic examples: factorial, sum of digits, power and simple sequences.

Recursion and iteration are alternative control techniques.

CO 3 — MODULE 3: ARRAYS & ARRAY PROCESSING

1-D ARRAYS

Declaration → instantiation → initialization

Example:
boolean[] occupied = new boolean[20];

Use indexing and length.

Defensive iteration prevents bounds errors.

ARRAY PATTERNS

- Traversal
- Accumulation
- Linear search
- Finding extremes
- Counting
- Reversing

These patterns help manage room and occupancy data.

2-D & METHODS

2-D arrays: rows, columns and nested loops for tables/grids.

Arrays can be passed to and returned from methods.

Arrays are reference types, so aliasing must be handled carefully.

Math methods can be used as black-box abstractions.

SYSTEM WORKFLOW — INTEGRATING CO 1, CO 2 & CO 3

1 **START**

Java program begins

2 **MENU**

Show available operations

3 **SELECT**

switch handles user choice

4 **CHECK**

if-else checks room status

5 **BOOK/CANCEL**

methods update room data

6 **OCCUPANCY**

loops count room status

7 **BILL**

arithmetic calculates total

Module 1 defines the problem • Module 2 controls decisions/repetition • Module 3 organizes reusable code and data.

SAMPLE OUTPUT & OCCUPANCY CALCULATION

```
==============================
    HOTEL BOOKING MANAGER
==============================

1. View Rooms
2. Book Room
3. Cancel Booking
4. Check Occupancy
5. Exit

Enter choice: 2
Customer: Rahul
Room: 105
Days: 3

Room booked successfully!
Room Price: ₹2500/day
Total Bill: ₹7500
```

OCCUPANCY

Total rooms = 20
Occupied rooms = 15
Available rooms = 5

FORMULA

Occupancy Rate =
(Occupied / Total) × 100
= (15 / 20) × 100 = 75%

BILLING

₹2500 × 3 days = ₹7500

GitHub Project Link

https://github.com/rachuriharshini/KLH-FED-SECTION16-PSPJTEAM18-new.git

CONCLUSION, FUTURE SCOPE & THANK YOU

CONCLUSION

- Manage hotel rooms
- Handle customer bookings
- Track occupancy
- Calculate bills
- Reduce manual errors

FUTURE SCOPE

- Add a database
- Java Swing/JavaFX GUI
- Online payment
- Customer login
- Booking receipts

LEARNING OUTCOME

CO1: Think and specify the problem
CO2: Apply selection and iteration
CO3: Build reusable methods and array-based processing

Project: Hotel Booking & Occupancy Management System

THANK YOU!